PÓS TECH

DEVOPS E ARQUITETURA CLOUD

FASE 3 | AULA 02 -

OTIMIZAÇÃO E ESCALABILIDADE DE PIPELINES

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

SUMÁRIO

O QUE VEM POR AÍ? 3
HANDS ON 4
SAIBA MAIS 5
MERCADO, CASES E TENDÊNCIAS 12
O QUE VOCÊ VIU NESTA AULA? 14
REFERÊNCIAS 15

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

O QUE VEM POR AÍ?

Na era da computação em nuvem, a entrega de software ágil e segura é fundamental. A automação de pipelines de CI/CD é o nosso ponto de partida. Nesta aula, vamos além do básico, explorando como otimizar pipelines complexos para reduzir o tempo de execução e implementar estratégias de escalabilidade que garantam governança e reutilização em toda a organização.

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

HANDS ON

As videoaulas irão guiá-lo(a) em uma jornada prática. Na primeira, você vai reestruturar um pipeline monolítico para o formato multi-stage. Em seguida, na segunda videoaula, usará a matrix strategy para otimizar o tempo de execução com paralelismo. Finalmente, a terceira videoaula abordará a criação de workflows reutilizáveis, essenciais para governança e padronização. Use este material para aprofundar o que será visto nos vídeos!

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

SAIBA MAIS

A otimização e a escalabilidade de pipelines de CI/CD são a evolução natural da automação para o ambiente corporativo. Enquanto um pipeline simples é suficiente para um único projeto ou uma pequena equipe, a complexidade e a ineficiência se tornam barreiras significativas à medida que a organização cresce. O papel de um(a) líder técnico(a) é antecipar esses desafios e estruturar os fluxos de trabalho de automação para que eles não apenas funcionem, mas também sejam sustentáveis e eficientes em escala.

A Estrutura Lógica: Orquestrando Pipelines Multi-Stage

A otimização de um pipeline começa com a sua estrutura. Em vez de um único e longo script, um workflow deve ser visto como uma série de jobs com propósitos específicos. Essa abordagem, conhecida como pipeline multi-stage, divide o processo de entrega de software em fases lógicas e independentes, como build, test e deploy.

A grande vantagem dessa modularização é a clareza e a facilidade de gerenciamento. Se um dos estágios falha, como por exemplo o estágio de teste de integração, os estágios subsequentes de implantação nem sequer são iniciados e o feedback é direcionado de forma precisa para a causa-raiz do problema. A orquestração desses estágios é realizada de forma declarativa utilizando a sintaxe jobs.<job_id>.needs. O campo needs define uma dependência entre os jobs, garantindo uma sequência lógica de execução. Por exemplo, o job de test pode declarar needs: build, assegurando que ele só iniciará após a conclusão bem-sucedida do job de build.

Essa abordagem de dependência explícita é fundamental para construir pipelines robustos e confiáveis, pois a qualidade do código aumenta à medida que ele avança por cada "portão" de validação no fluxo de trabalho.

Maximizando a Eficiência: O Poder do Paralelismo com Matrix Strategy

A otimização do tempo de execução é um dos principais desafios em pipelines de alta complexidade. Um fluxo de trabalho que testa uma aplicação em várias versões de um framework ou em diferentes sistemas operacionais pode se tornar um gargalo significativo. A estratégia de matriz (matrix strategy) é a resposta

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

do GitHub Actions para esse problema. Essa funcionalidade permite a execução de múltiplos jobs em paralelo a partir de uma única definição de pipeline, eliminando a duplicação de código e acelerando o feedback para os desenvolvedores.

A estratégia de matriz é definida no bloco strategy.matrix de um job, em que você especifica variáveis e uma lista de valores para cada uma delas. O GitHub Actions, então, criará e executará um job para cada combinação possível dessas variáveis. Por exemplo, para testar uma aplicação Node.js em várias versões do runtime e em diferentes sistemas operacionais, a matriz pode incluir version: e os: [ubuntu-latest, windows-latest]. Isso resultará na execução de quatro jobs em paralelo, um para cada combinação, todos definidos a partir de um único bloco de código.

Além da sua simplicidade, a estratégia de matriz oferece funcionalidades avançadas para refinar o paralelismo. O uso das palavras-chave include e exclude permite adicionar ou remover combinações específicas da matriz, útil para casos de uso especiais ou para excluir combinações incompatíveis. A funcionalidade fail-fast: true (habilitada por padrão) otimiza o consumo de recursos ao cancelar todos os jobs em andamento se um deles falhar, economizando tempo e custo. No entanto, é possível desativá-la com fail-fast: false se você precisar que todos os jobs da matriz sejam concluídos, independentemente de falhas individuais. Você também pode controlar o número de jobs paralelos com max-parallel para gerenciar a carga nos seus runners.

Aprofundando em Estratégias de Matriz: O Poder da Dinâmica com fromJSON()

Enquanto a estratégia de matriz estática é poderosa, cenários mais avançados, como em monorepos, exigem a capacidade de gerar a matriz dinamicamente em tempo de execução. Em vez de testar todo o código base em todas as combinações, um job anterior pode analisar as mudanças e gerar uma lista precisa de tarefas. A chave para essa funcionalidade no GitHub Actions é a função fromJSON(), que transforma uma string JSON em uma matriz. Essa abordagem permite criar um workflow que primeiro filtra quais jobs são necessários e depois gera a matriz dinamicamente para os jobs subsequentes, como um conjunto de testes relevantes, economizando tempo e recursos de forma significativa.

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

Por exemplo, um job pode usar um script para listar os diretórios alterados em um pull request, converter essa lista em uma string JSON e passá-la como output para um job subsequente. Este job receptor, por sua vez, usa a função fromJSON() para consumir a lista e criar um conjunto de jobs em paralelo, um para cada diretório ou projeto que precisa ser testado ou compilado. Isso é o que separa um pipeline eficiente de um pipeline que consome recursos de forma desnecessária, sendo uma estratégia fundamental para otimização em larga escala.

Governança e Reutilização: A Chave para a Escalabilidade em Organizações

A escalabilidade em CI/CD vai além da performance; ela é sobre a capacidade de padronizar processos de forma consistente em toda a organização. Sem um mecanismo de reutilização, cada nova equipe ou projeto pode "reinventar a roda", criando pipelines que diferem em qualidade, segurança e conformidade.

Os workflows reutilizáveis (reusable workflows) no GitHub Actions resolvem esse problema, atuando como a ferramenta principal para centralizar a lógica de automação. Um workflow reutilizável é um template que pode ser chamado por outros workflows (os workflows "chamadores" ou callers). Ele é definido com o gatilho on: workflow_call, que especifica os inputs e secrets que podem ser passados para ele.

Isso permite que um único workflow reutilizável, por exemplo, um que lida com o deploy de uma aplicação em ambiente de produção, seja mantido em um repositório centralizado e chamado por todos os repositórios da organização. As alterações e as melhorias nesse workflow central se aplicam automaticamente a todos os pipelines que o utilizam, garantindo consistência, reduzindo o esforço de manutenção e promovendo as melhores práticas em toda a empresa.

A escolha da ferramenta certa para a reutilização é crucial para o(a) líder técnico(a). Existe uma distinção importante entre workflows reutilizáveis e as Ações Compostas (Composite Actions), que, embora pareçam similares, têm escopos e capacidades muito diferentes.

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

Característica	Workflows Reutilizáveis	Ações Compostas (Composite Actions)
Nível de Reutilização	Reutiliza um workflow inteiro, com múltiplos jobs e steps.	Reutiliza um conjunto de steps dentro de um único job.
Escopo	Pode ser chamado de jobs em workflows diferentes (inclusive em outros repositórios).	É executado como um step dentro de um job em um único workflow.
Acesso a Segredos	Permite o acesso a secrets passados pelo workflow chamador via workflow_call.	Não pode acessar segredos diretamente, a menos que sejam passados como inputs, o que representa uma limitação.
Visibilidade de Logs	Oferece um log detalhado para cada job e step individualmente.	Gera um único log para todos os steps agrupados, o que pode dificultar a depuração.

Tabela 1 – Workflows Reutilizáveis vs. Composite Actions
Fonte: elaborado pelo autor (2025)

A compreensão das diferenças entre essas duas ferramentas é fundamental para o(a) líder técnico(a). Por exemplo, a incapacidade das Ações Compostas de acessarem segredos de forma segura limita sua aplicação em cenários de implantação, em que credenciais sensíveis são necessárias. Os workflows reutilizáveis, por sua vez, oferecem uma solução robusta e segura, ideal para centralizar e padronizar processos complexos de CI/CD que envolvem segredos.

Adoção Estratégica: A Reutilização como a Base da Engenharia de Plataformas

O conceito de workflows reutilizáveis se conecta diretamente com uma das tendências mais importantes do mercado: a **Engenharia de Plataformas** (Platform

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

Engineering). O DevOps, como uma filosofia cultural, enfatiza a automação e a colaboração, mas não oferece uma solução para o "crescimento caótico" de toolchains e processos. A Engenharia de Plataformas surge como a próxima evolução, focando na construção de plataformas internas de autoatendimento (Internal Developer Platforms - IDPs) que padronizam e simplificam o ciclo de vida do software.

A criação de um repositório central de workflows é a materialização prática da Engenharia de Plataformas. Nesse modelo, uma equipe de plataforma centraliza e "produtiza" os processos de automação, como um workflow reutilizável para o deploy de uma aplicação ou para a análise de segurança estática do código. Os desenvolvedores, que são os "clientes" da plataforma, podem então consumir esses workflows testados e aprovados, focando na lógica de negócio e na criação de valor sem se preocupar com as complexidades da infraestrutura e do toolchain. Isso não apenas promove a padronização e a governança, mas também capacita as equipes de desenvolvimento, transformando a equipe de DevOps de um papel de suporte reativo para um de "habilitador" estratégico.

Governança e Segurança em Escala: Protegendo o Repositório Central de Workflows

Para um(a) líder técnico(a), a governança e a segurança de um repositório central de workflows são cruciais. A adoção de políticas de segurança rigorosas é essencial para garantir a integridade dos workflows reutilizáveis. Uma prática recomendada é sempre referenciar o workflow central usando um commit SHA completo em vez de uma tag ou um branch. Isso garante que o caller workflow sempre execute exatamente a mesma versão de código, evitando o risco de que alterações não autorizadas ou inseguras sejam introduzidas.

A passagem de segredos (secrets) para workflows reutilizáveis deve ser gerenciada com cuidado, utilizando os mecanismos nativos do GitHub Actions para garantir que dados sensíveis sejam transmitidos com segurança e apenas quando necessário. Políticas de segurança baseadas no princípio do menor privilégio (least privilege) devem ser aplicadas, concedendo permissões apenas para o que é estritamente necessário para a execução do pipeline.

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

A auditoria regular de como os segredos são tratados e a rotação periódica de segredos registrados também são medidas essenciais para manter o ambiente seguro e reduzir o risco de credenciais comprometidas. Ao implementar essas medidas, o repositório central de workflows se torna um pilar de segurança e conformidade, não um vetor de risco.

Runners: Onde o Código Ganha Vida

Um runner é o ambiente de execução no qual seus jobs de workflow operam. O GitHub Actions oferece dois tipos principais: os GitHub-hosted runners e os self-hosted runners.

Os **GitHub-hosted runners** são ambientes de máquina virtual gerenciados e hospedados pelo GitHub, prontos para uso. Eles oferecem uma experiência zero-configuration, o que os torna ideais para projetos open source e equipes que priorizam a agilidade. No entanto, eles vêm com uma cota de minutos gratuitos e têm custos associados para uso em repositórios privados.

A escolha estratégica de runners se torna crucial ao considerar otimização, segurança e custo. É aqui que os **self-hosted runners** entram em cena. Ao contrário dos runners gerenciados, os self-hosted runners são máquinas virtuais ou físicas que você mesmo provisiona e mantém. Eles oferecem dois grandes benefícios que justificam o esforço de gerenciamento:

- **Custo**: em repositórios privados, os minutos de execução de self-hosted runners não são cobrados pelo GitHub. Embora haja o custo de infraestrutura (servidores, energia), em projetos de larga escala com milhares de minutos de execução, essa pode ser uma economia significativa.
- **Segurança e Controle**: o self-hosted runner oferece controle total sobre o ambiente de execução. Isso permite que você aplique políticas de segurança corporativas mais rígidas, utilize imagens de sistema operacional hardened e controle o acesso à rede de forma granular. Além disso, para deployments na nuvem, o uso de self-hosted runners em uma rede privada reduz a necessidade de expor credenciais de alto privilégio para fora do ambiente de nuvem, pois o runner pode herdar permissões de

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

um IAM Role específico da máquina. Essa abordagem de least privilege é um pilar da segurança em nuvem.

No entanto, há uma grande ressalva de segurança: self-hosted runners não são recomendados para repositórios públicos, pois o código de qualquer pull request pode ser executado em sua infraestrutura, abrindo um vetor de ataque perigoso. Outra consideração importante é o gerenciamento da escalabilidade. Embora os GitHub-hosted runners sejam escaláveis por padrão, a escalabilidade dos self-hosted runners requer automação adicional, como o Actions Runner Controller (ARC), para criar e destruir runners efêmeros conforme a demanda.

Maximizando o Desempenho: Artifacts e Cache

Além das estratégias de paralelismo, o(a) líder técnico(a) deve otimizar o tempo de execução dos pipelines por meio do gerenciamento eficiente de artefatos e do uso de cache de dependências. Artefatos são os arquivos criados durante o build (como um arquivo jar, uma imagem Docker ou um executável) que precisam ser compartilhados entre os jobs do workflow ou armazenados para uso posterior.

Para o hands on da videoaula, a transferência do artefato (artifact) do job de build para o job de test é um exemplo fundamental de como o gerenciamento de artefatos opera. O GitHub Actions fornece uma maneira simples de fazer isso, garantindo que o job de test opere sobre o mesmo artefato produzido pelo build, mantendo a consistência do fluxo de trabalho.

O cache de dependências (dependency caching) é uma técnica poderosa para acelerar o tempo de execução de um workflow, evitando que as dependências (como os pacotes npm, pip ou Maven) sejam baixadas e instaladas a cada nova execução. O GitHub Actions oferece um recurso nativo de cache que salva as dependências em um diretório e as restaura automaticamente em execuções subsequentes. Ao configurar, a pessoa desenvolvedora define uma chave para identificar o conteúdo do cache.

Se essa chave for a mesma em uma nova execução, o cache é restaurado e o tempo de instalação é drasticamente reduzido. Essa técnica é especialmente benéfica em projetos com um grande número de dependências, permitindo que desenvolvedores(a) tenham um feedback mais rápido e otimizando os custos de execução.

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

MERCADO, CASES E TENDÊNCIAS

A excelência na otimização e na escalabilidade de pipelines de CI/CD é um diferencial competitivo no mercado. Casos de sucesso globais demonstram que a automação não é apenas uma prática técnica, mas um pilar estratégico que pode impulsionar a inovação e o crescimento de negócios. A Netflix, por exemplo, utilizou a automação de seus pipelines para gerenciar uma arquitetura massiva de microsserviços, o que foi crucial para sua transição de um serviço de aluguel de DVDs para um dos maiores produtores de conteúdo global.

De forma similar, a Amazon construiu uma infraestrutura de TI robusta para suas operações de e-commerce e, ao perceber o valor de sua automação, transformou essa capacidade ociosa em um novo modelo de negócio: a Amazon Web Services (AWS). Esses casos mostram que a automação e a otimização de pipelines se traduzem diretamente em resultados de negócio e liderança de mercado.

A ascensão da Engenharia de Plataformas é uma das tendências mais significativas que moldam o futuro do CI/CD. Conforme os princípios de DevOps se tornaram a norma, as organizações de maior porte passaram a enfrentar um novo desafio: a complexidade e a fragmentação das ferramentas e processos.

A Engenharia de Plataformas, vista por especialistas como a próxima evolução do DevOps, aborda essa questão de forma proativa, construindo plataformas internas de autoatendimento (Internal Developer Platforms - IDPs) que simplificam o toolchain e reduzem a sobrecarga cognitiva de especialistas em desenvolvimento. O mercado em 2025 já vê um movimento significativo para a adoção dessa filosofia, com o objetivo de institucionalizar as melhores práticas e acelerar a entrega de valor em escala.

Outra tendência emergente é a integração da Inteligência Artificial (IA) nos pipelines de CI/CD. A IA já está sendo utilizada para otimizar os fluxos de trabalho, predizer falhas de build antes que ocorram, identificar testes flaky (instáveis) e alocar recursos de forma dinâmica. Ferramentas impulsionadas por IA podem analisar o histórico de execução de pipelines para sugerir melhorias, ou até mesmo

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

automatizar decisões, como quando implantar uma nova versão em produção, com base em métricas de desempenho e risco.

Embora ainda em fase inicial, a integração da IA sinaliza um futuro em que os pipelines serão não apenas automatizados, mas também autônomos e preditivos. Para um(a) líder técnico(a), estar atento a essas tendências é essencial para posicionar a equipe e a organização na vanguarda da entrega de software.

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, desmistificamos o conceito de otimização e escalabilidade de pipelines. O pipeline, que em sua forma mais simples é apenas um fluxo de automação, foi apresentado como um ativo estratégico para a organização.

Exploramos a importância de estruturar workflows em estágios lógicos (multi-stage) para clareza e controle e o poder do paralelismo com a matrix strategy para maximizar a eficiência e reduzir o tempo de feedback.

Finalmente, entendemos como a reutilização de workflows é a chave para a governança e a padronização, ligando essa prática diretamente à ascensão da Engenharia de Plataformas.

O objetivo é que você transforme um pipeline funcional em uma espinha dorsal de engenharia que impulsiona a produtividade, a padronização e a inovação em escala.

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

REFERÊNCIAS

CODEFRESH. **How to use GitHub Actions Matrix**. 2025. Disponível em: https://codefresh.io/learn/github-actions/github-actions-matrix/. Acesso em: 07 out. 2025.

DEV.TO. **Composite Actions vs. Reusable Workflows: What is the Difference?**. 2023. Disponível em: https://dev.to/n3wt0n/composite-actions-vs-reusable-workflows-what-is-the-difference-github-actions-11kd. Acesso em: 07 out. 2025.

GITHUB. **GitHub Actions documentation**. 2025. Disponível em: https://docs.github.com/actions. Acesso em: 07 out. 2025.

INCREDIBUILD. **Best practices to create reusable workflows on GitHub Actions**. 2025. Disponível em: https://www.incredibuild.com/blog/best-practices-to-create-reusable-workflows-on-github-actions. Acesso em: 07 out. 2025.

MICROSOFT. **O que é engenharia de plataforma?**. 2024. Disponível em: https://learn.microsoft.com/pt-br/platform-engineering/what-is-platform-engineering. Acesso em: 07 out. 2025.

RED HAT. **Platform engineering vs. DevOps**. 2025. Disponível em: https://www.redhat.com/pt-br/topics/platform-engineering/platform-engineering-vs-devops. Acesso em: 07 out. 2025.

SAINI, A. **Continuous Integration & Deployment Best Practices To Look Forward In 2025**. 2025. Disponível em: https://www.kellton.com/kellton-tech-blog/continuous-integration-deployment-best-practices-2025. Acesso em: 07 out. 2025.

TROTTA, F. **Mastering AI-Enhanced CI/CD Pipelines for Optimal Software Delivery**. 2025. Disponível em: https://zencoder.ai/blog/building-ai-enhanced-ci-cd-pipelines-for-enterprise-applications. Acesso em: 07 out. 2025.

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

PALAVRAS-CHAVE

CI/CD. DevOps. Otimização de Pipelines. Escalabilidade. GitHub Actions. Matrix Strategy. Workflows Reutilizáveis. Governança. Paralelismo. Engenharia de Plataformas.

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com



PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com